FinSync Platform Architecture Guide

Comprehensive Reference for Basic & Intermediate Java, Spring Boot & React Concepts

Section 1: Executive System Architecture & Tech Stack

FinSync is a modern, enterprise-grade digital banking and wealth management application. It provides real-time multi-account handling, P2P fund transfers, expense tracking analytics, automated savings goals, and bank-grade security protocols.

Core Technology Stack Breakdown

- **Backend Layer:** Built with Java 17/21 & Spring Boot 3.2. Provides REST API endpoints, Spring Data JPA persistence, Spring Security with JWT, and transactional safety.
- **Frontend Layer:** Built with React 18, Vite build tool, React Router DOM, Axios HTTP client, and custom glassmorphic Vanilla CSS components.
- **Database Layer:** Relational database architecture configured for MySQL (jdbc:mysql://127.0.0.1:3306/finsync_db) with embedded H2 database fallback capability.
- **Security Architecture:** Stateless JWT (JSON Web Token) bearer authentication with BCrypt password hashing and CORS/CSRF protection.

Section 2: Basic Java & Spring Boot Fundamentals

The FinSync backend uses Spring Boot to minimize boilerplate configuration and enable rapid, robust API development.

1. Spring Annotations & Dependency Injection (DI)

- **@SpringBootApplication:** The entry point annotation combining @Configuration, @EnableAutoConfiguration, and @ComponentScan.
- **@RestController & @RequestMapping:** Marks Java classes as HTTP REST controllers that automatically serialize return values to JSON format.
- **@Service & @Repository:** Stereotype annotations identifying service layer business logic and database access objects.
- **@RequiredArgsConstructor:** Lombok annotation generating constructors for final fields, enabling clean constructor-based dependency injection without manual wiring.

Sample REST Controller Implementation

```
@RestController
@RequestMapping("/api/accounts")
@RequiredArgsConstructor
public class AccountController {

    private final AccountService accountService;
    private final CurrentUser currentUser;

    @PostMapping
    public ResponseEntity<?> createAccount(@Valid @RequestBody CreateAccountRequest req) {
        return ResponseEntity.ok(accountService.createAccount(currentUser.id(), req));
    }

    @GetMapping
    public ResponseEntity<?> getMyAccounts() {
        return ResponseEntity.ok(accountService.getUserAccounts(currentUser.id()));
```

}

}

2. DTO Validation & Bean Constraints

To enforce data integrity and prevent malformed data from reaching the database, incoming JSON payloads are validated using Jakarta Bean Validation annotations:

- **@NotBlank & @NotNull:** Ensures required string fields are non-empty and required numeric/object fields are non-null.
- **@Email Validation:** Enforces strict RFC-compliant email formatting on user registration and login endpoints.

```
public class RegisterRequest {
    @NotBlank
    public String fullName;

    @NotBlank
    @Email
    public String email;

    @NotBlank
    public String password;

    public String phoneNumber;
}
```

Section 3: Intermediate Backend & Security Concepts

1. JPA/Hibernate Relational Entity Mapping

Database tables are mapped to Java domain objects using Java Persistence API (JPA) annotations.

- **@Entity & @Table:** Defines the Java class as a persistent database entity bound to a specific SQL table name (e.g. @Table(name = "accounts")).
- **@ManyToOne Relational Mapping:** Defines foreign key relationships (e.g. many bank accounts belong to one User) using @JoinColumn(name = "user_id").
- **@PrePersist Lifecycle Hooks:** Executes custom code automatically right before an entity is inserted into the database, such as initializing creation timestamps (LocalDateTime.now()).

```
@Entity
@Table(name = "accounts")
@Getter @Setter @NoArgsConstructor
public class Account {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String accountNumber;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @Enumerated(EnumType.STRING)
    private AccountType accountType;

    @Column(nullable = false, precision = 15, scale = 2)
    private BigDecimal balance = BigDecimal.ZERO;
}
```

2. ACID Financial Transactions & Double-Entry Bookkeeping

Money movements require absolute reliability and atomicity to prevent data corruption or lost funds.

- **Declarative @Transactional:** Ensures database mutations execute as an indivisible unit. If an exception occurs, all changes automatically rollback.
- **READ_COMMITTED Isolation:** Prevents dirty reads during concurrent balance operations across accounts.
- **Double-Entry Bookkeeping:** Transfers debit the source account and credit the destination account while simultaneously logging matching ledger records (TRANSFER_OUT and TRANSFER_IN).

```
@Override
@Transactional(isolation = Isolation.READ_COMMITTED, rollbackFor = Exception.class)
public Map<String, Object> transfer(Long userId, TransferRequest req) {
    Account from = accountRepository.findByAccountNumber(req.fromAccountNumber)
            .orElseThrow(() -> new ResourceNotFoundException("Source account not found"));
    Account to = accountRepository.findByAccountNumber(req.toAccountNumber)
            .orElseThrow(() -> new ResourceNotFoundException("Recipient account not found"));

    if (from.getBalance().compareTo(req.amount) < 0) {
        throw new InsufficientBalanceException("Insufficient balance");
    }

    from.setBalance(from.getBalance().subtract(req.amount));
    to.setBalance(to.getBalance().add(req.amount));

    accountRepository.save(from);
    accountRepository.save(to);
    return Map.of("message", "Transfer completed successfully");
}
```

3. Spring Security & Stateless JWT Authentication

FinSync utilizes JSON Web Tokens (JWT) for secure, stateless session management without storing session state on the server.

- **BCrypt Password Hashing:** Passwords are never stored in plain text. Spring Security's BCryptPasswordEncoder applies salted one-way hashing.
- **Bearer Token Flow:** Upon successful login, the server signs a JWT containing the user ID and role. The client sends this token in the Authorization: Bearer <token> HTTP header for all subsequent requests.

Section 4: Frontend React Integration & Validation

The frontend is designed with React modular components and client-side validation to provide an intuitive user experience.

1. Realtime Client-Side Input Validation

- **Email Format Regex:** Validates input with /^[^s@]+@[^s@]+.[^s@]+$/ before triggering network calls.
- **Mobile Number Regex:** Enforces numeric phone formatting with at least 10 digits (/^[0-9+-\s()]{10,15}$/).
- **Autofill Prevention:** Disables browser auto-fill on registration forms using autoComplete="off" and autoComplete="new-password".

2. Axios Interceptors & Session Handling

- **Request Interceptor:** Automatically injects the stored JWT token from localStorage into outgoing HTTP headers.

- **Response Interceptor:** Catches 401 Unauthorized errors to automatically purge expired tokens and redirect users to the login screen.

Section 5: Relational Database Schema Table Summary

The MySQL database schema finsync_db consists of 5 core tables with defined primary and foreign keys:

- **users:** Columns: id (PK), email (UNIQUE), password, full_name, phone_number, role, created_at
- **accounts:** Columns: id (PK), user_id (FK -> users.id), account_number (UNIQUE), account_type, balance
- **transactions:** Columns: id (PK), account_id (FK -> accounts.id), type, amount, balance_after, counterparty_account_number, description, created_at
- **expenses:** Columns: id (PK), user_id (FK -> users.id), amount, category, note, expense_date
- **savings_goals:** Columns: id (PK), user_id (FK -> users.id), goal_name, target_amount, saved_amount, achieved